Technische Beschreibung

Aufbau

Teil	Technik
Server	Python 3.12, FastAPI, Uvicorn; läuft als Hintergrundprozess auf einem Windows-PC
Datenbank	SQLite (eine Datei `daten/lager.db`, WAL-Modus), Zugriff über SQLAlchemy Core
Oberfläche	Server-gerenderte Seiten (Jinja2), HTMX, Alpine.js, Tailwind CSS; alle Dateien lokal, kein Internet nötig
Scanner-App	`/m`, installierbare Web-App (Manifest, Service Worker `/m/sw.js`); Scantaste über Tastatureingabe (DataWedge), Kamera-Scan über BarcodeDetector bzw. ZXing (lokal)
Offline-Modus	Warteschlange in IndexedDB des Geräts, Artikelkatalog `/m/katalog.json` zwischengespeichert, Übertragung über `/m/sync` mit UUID je Buchung (Tabelle `sync_log` verhindert Doppelbuchungen)
HTTP / HTTPS	HTTP (Port 8080) lauscht nur auf `127.0.0.1` – für den Browser am Lager-PC. Alle anderen Geräte nur über HTTPS (Port 8443). Abschaltbar mit `http_nur_lokal = false` (nicht empfohlen).
Zertifikate	Eigene kleine CA (`daten/lager_ca.crt`, Download `/zertifikat.crt`; der private Schlüssel `daten/lager_ca_schluessel.pem` wird nie ausgeliefert). Die CA ist per *Name Constraints* (kritisch) auf 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16 und die Namen dieses PCs beschränkt – auch mit gestohlenem Schlüssel lassen sich keine Zertifikate für fremde Domains ausstellen. Das Serverzertifikat wird beim Start erneuert, wenn sich IP/PC-Name geändert haben oder es in weniger als 30 Tagen abläuft; öffentliche IP-Adressen werden nicht aufgenommen.
Etikettendruck	TSPL direkt an HPRT HT100 (Windows-Spooler RAW über winspool.drv oder TCP 9100); Fallback Browserdruck
Sicherung	ZIP mit konsistenter SQLite-Kopie (Backup-API) und Anhängen; mehrere Uhrzeiten am Tag, mehrere Speicherorte, nur bei Änderung, Wiederherstellen im laufenden Betrieb (`app/services/sicherung.py`)

Gleichzeitige Buchungen (PC, mehrere Handscanner, API) werden über Schreibtransaktionen mit sofortiger Sperre (`BEGIN IMMEDIATE`) nacheinander ausgeführt; Lesen blockiert nicht. Getestet mit parallelen Entnahmen des letzten Teils (Threads und echte HTTP-Anfragen).

Anmeldung: Passwörter mit mindestens 10 Zeichen, gespeichert als PBKDF2-SHA256 (240.000 Runden). Nach 5 Fehlversuchen je Konto und Gerät bzw. 20 je Gerät ist die Anmeldung 15 Minuten gesperrt (nur im Speicher; Neustart hebt die Sperre auf). Der erste Administrator kann nur am Lager-PC selbst angelegt werden. Die Sitzung liegt signiert im Cookie (30 Tage; über HTTPS mit `Secure`, immer `HttpOnly`, `SameSite=Lax`), wird aber bei jedem Aufruf gegen die Tabelle `users` geprüft – Sperre, Rollenwechsel und Passwortänderung wirken sofort. Größere Daten (Sammelentnahme-Liste, Übernahmebericht) liegen in der Tabelle `settings`, nicht im Cookie (Browsergrenze 4 KB). Weiterleitungen (`weiter`, Referer) sind auf Pfade der eigenen Anwendung beschränkt; Anhänge werden nur als Bild/PDF/Text im Browser angezeigt, alles andere als Download.

Module, Rechte, Migrationen (ab 2.5.0)

- **Zwei Bereiche, ein Programm, eine Datenbank:** Lagerverwaltung (`/`, `/m`) und Instandhaltung (`/ih`, Modul `app/routes/ih.py`). Die Instandhaltung ist mit `[cmms] aktiv` in `config.toml` abschaltbar (Standard: aus). Ihre Tabellen heißen `ih_…`. Das Layout (`base.html`) wählt Navigation und Kopf nach dem Pfad (`bereich` = `lager` | `ih`).
- **Rechte:** Die Lager-Rolle steht weiter in `users.rolle`, mit der Reihenfolge `keine < lesen < lager < admin`. Unbekannte Werte haben keinen Zugriff. Die Rolle für die Instandhaltung steht in `benutzer_rollen` (Benutzer, Modul `ih`, Rolle). Was eine Rolle darf, ist in `app/rechte.py` als Liste von Rechten festgelegt (z. B. `ih.import`). Jede CMMS-Route prüft ihr Recht mit `require_recht()` auf dem Server. `tests/test_cmms_phase0.py` ruft dafür jede Route mit jeder Rolle auf. Lager-Administratoren haben alle CMMS-Rechte. Die API lehnt Benutzer ohne Lagerrolle ab.
- **Migrationen** (`app/migrationen.py`, Tabelle `schema_version`): Sie sind nummeriert, wiederholbar und fügen nur hinzu (neue Tabellen, neue optionale Spalten). Alle ausstehenden Migrationen laufen in einer Transaktion. Beim Start (`vor_start`) passiert erst ein **Probelauf an einer Kopie** der Datenbank (SQLite-Backup-API). Danach müssen Artikel, Plätze, Bestand je Artikel und Platz (Prüfsumme), Anzahl und Summe der Buchungen, Benutzer und Reservierungen unverändert sein, und `PRAGMA integrity_check` muss `ok` melden. Erst dann wird `vor_update_….zip` im Sicherungsordner abgelegt (ersatzweise `daten/vor_update_….db`) und die echte Datenbank aktualisiert. Schlägt der Probelauf fehl, startet das Programm nicht, und die Datenbank bleibt unberührt. Auch beim Wiederherstellen einer älteren Sicherung laufen die Migrationen. Migration 1 ergänzt `bewegungen.preis` (Artikelpreis zum Buchungszeitpunkt, für Kosten je Anlage), `bewegungen.auftrag_id` und `reservierungen.auftrag_id` (für die Aufträge der nächsten Ausbaustufe).
- **Wartungsplanung** (ab 2.6.0, `app/services/wartung.py`, Seiten in `app/routes/ih_wartung.py`):
  - `plan_lesen()` liest die Blätter *Maßnahmen*, *Intervalle je Klasse*, *Kampagnen* und *Pflichtprüfungen* (Spalten über den Namen, ohne Rücksicht auf Groß-/Kleinschreibung und Sonderzeichen). Pflichtprüfungen ohne eigene Maßnahmenzeile werden als Maßnahme ergänzt. Der Plan liegt bis zur Übernahme als JSON in `ih_import_zeilen` (Lauf mit `art = plan`).
  - Übernahme nach Schlüssel: `ih_massnahmen.code`, `ih_klassen_intervalle.schluessel` (ID), `ih_kampagnen.schluessel` (GruppenID). Vorhandene Einträge werden aktualisiert, nichts wird gelöscht.
  - Termine: `startpunkt + n × intervall_monate`, mit Monatsende-Korrektur. `startpunkt()` ist der erste Termin (`rhythmus = fest`, Standard) oder bei `rhythmus = ab_erledigung` der Montag von „letztes `erledigt_am` eines abgeschlossenen Auftrags + Intervall“, nie vor dem ersten Termin (`basis_je_kampagne()`). Ein Termin ist erledigt, wenn sein Auftrag abgeschlossen oder storniert ist. Lage: überfällig, wenn Termin + Dauer ≤ heute ohne Abschluss; läuft; bald (im Vorlauf); geplant.
  - Bauteile einer Kampagne: aktive Anlagen der Bauteilklasse. `bauteilklasse = Anlage` bedeutet ohne Bauteilliste. Eigene Kampagnen (ab 2.7.0, `eigen`, Schlüssel `E-…`) können stattdessen einzelne Bauteile haben (`ih_kampagne_bauteile`); `bauteile_kampagne()` nimmt dann nur diese. `taetigkeit` beschreibt, was zu tun ist. Die Planübernahme setzt `rhythmus`, `taetigkeit` und `eigen` nicht.
  - Ersatzteile (ab 2.7.0): `ih_stueckliste` (Bauteil ↔ Lagerartikel, Menge je Tausch, eindeutig je Paar) – gepflegt am Artikel (`tags_speichern()` in `app/routes/artikel.py`, nur wenn das Formular den Abschnitt enthält) oder an der Anlage. `vorschlaege_aus_verwendung()` sucht Tag-Nummern im Freitext `artikel.verwendung`

(Leerzeichen egal, nicht als Teil längerer Nummern). Entnahme für einen Auftrag: `POST /ih/auftraege/{id}/entnahme` bucht über `Lager.ausgang()` mit `auftrag_id`, Empfänger „Auftrag A-…“, Quelle `ih`; Kosten = Menge × `bewegungen.preis`.
    - Aufträge (`ih_auftraege`, eindeutig je Kampagne und Termin) mit Positionen je Bauteil (`ih_auftrag_positionen` = Protokoll). Formular: WP-02 nur für Sichtprüfung ohne Sicherheitsfunktion, sonst WP-01, bei Pflichtprüfungen eine Ergebnisliste.
    - Status: angelegt → freigegeben → in_arbeit (automatisch beim ersten Protokoll) → erledigt → abgeschlossen, dazu storniert und „zurück in Arbeit“. Regeln: `status_aendern()`, Verlauf in `ih_statuslog`.
    - Nachweise in `ih_dokumente` (Dateien `daten/anhaenge/ih_*`, in der Sicherung enthalten). Nur Bild, PDF und Text werden im Browser angezeigt, alles andere als Download.
    - Rechte: `ih.wartung.planen`, `ih.auftrag.freigeben`, `ih.auftrag.ausfuehren`, `ih.auftrag.abschliessen` (`app/rechte.py`). Migration 2 ergänzt `ih_import_laeufe.art`, Migration 3 (2.7.0) `ih_kampagnen.rhythmus`, `taetigkeit`, `eigen`.
- **Schichtbuch** (ab 2.8.0, `app/services/schichtbuch.py`, Seiten in `app/routes/schichtbuch.py`, Pfad `/ih/schichtbuch`): Tabellen `sb_teams`, `sb_zeiten` (Beginn/Ende als HH:MM, Ende ≤ Beginn = Folgetag), `sb_mitglieder` (Benutzer → Team), `sb_schichten` (eindeutig je Datum des Beginns und Schichtzeit, Team, Schichtführer, begonnen/beendet, Übergabenotiz), `sb_meldungen` (Felder der SharePoint-Liste, Asset als `platz_id` + Momentaufnahme der Tag-Nr., Work Order als Text + `auftrag_id`, Übergabe/Übernahme/Erledigt jeweils mit Schicht) und `sb_verlauf`. `slot()` bestimmt die laufende Schicht aus der Uhrzeit. Eine Meldung gehört zur laufenden Schicht (wird bei Bedarf angelegt). Übergabe: `uebergabe` + Text + übergebende Schicht; offen, solange `uebernommen_am` leer ist. Teams und Zeiten werden beim ersten Aufruf vorbelegt (Grün, Blau, Gelb, Rot; Tag/Nacht), Ereignistypen stehen in `settings` (`schichtbuch.ereignistypen`). Rechte: `ih.schichtbuch.ansehen`, `ih.schichtbuch.schreiben`, `ih.schichtbuch.einrichten`. Neue Tabellen, keine Migration nötig. Ab 2.9.0: `laufender_slot()` (die nächste Schicht zählt als laufend, sobald sie – bis 2 h vorher – begonnen wurde), `wieder_oeffnen()` (nur ohne begonnene Folgeschicht und ohne übernommene Übergaben), Übergaben sichtbar/übernehmbar nur aus beendeten Schichten, beim erneuten Beenden abgewählte Übergaben zurückgenommen (Verlauf `uebergabe_zurueck`). Nachtrag: `erstellt_am` = Ereigniszeitpunkt, `nachgetragen_am/von` (Migration 4), Schicht über `slot(zeitpunkt)`. Papierformulare `/ih/schichtbuch/papier`, `/papier/lager` (eigenes Druck-Gerüst `templates/papier/`), PDF-Fassungen in `docs/`.
- **Instandhaltungs-App** (ab 2.9.0, `app/routes/ih_app.py`): Startseite `/ih/m`, Manifest `/ih/manifest.webmanifest` (Scope `/ih/`, eigene Symbole `static/icon-ih-*`), Service Worker `/ih/sw.js` (Dateien vorhalten, ohne Verbindung Hinweis aufs Papierformular), `/ih/m/suche` (Tag-Nr. → Anlage). `base.html` verlinkt im Bereich Instandhaltung dieses Manifest und zeigt unter `lg` die Leiste unten.
- **Programmname** (`[server] programmname`, ab 2.9.0): Anmeldung, Titel, Desktop-Manifest, Arbeitsplatz-Skript, `windows/verknuepfung.ps1` (liest config.toml). Interne Namen (Ordner, Dienst, Dateien) bleiben.
- **Design** (ab 2.11.0, `[server] design` = `standard` | `ci`): `base.html` / `m/base.html` setzen die Klasse `ci` am `<html>`. `frontend/input.css` definiert unter `.ci` und `.ci.dark` die Farbvariablen (CI-Palette, geprüft auf WCAG-Kontrast: Text ≥ 4,5:1, Feldrahmen ≥ 3:1) und Komponenten-Anpassungen (Barlow 400/700, roter Akzentstreifen `.ci-kopf`, roter Balken am aktiven Menüpunkt, Info-Status in Schiefer `--info`, kritische Status hinterlegt). `font-cond` nutzt `var(--font-cond)` (Standard: Barlow Semi Condensed, CI: Barlow). Logo `static/logo-ci.svg` (rot) und `logo-ci-ih.svg` (navy), App-Symbole `icon-ci-*`, `icon-ci-ih-*`; die Manifeste wählen die Symbole nach dem Design. Schrift `fonts/barlow-latin-700-normal.woff2` (OFL, Fontsource).

- **Umzugspaket** (ab 2.10.0, `app/services/umzug.py`): erst Sperre setzen (`settings` `umzug.sperre`, `app.state.umzug_sperre`), dann stimmige DB-Kopie (SQLite-Backup, Sperr-Eintrag in der Kopie gelöscht), ZIP mit oberstem Ordner = Programmordnername: Programmdateien (ohne `daten`, `logs`, `backups`, `druckausgabe`, `testsystem`, Caches), `daten/` (Zertifikate, Anhänge), `config.toml` aus den laufenden Einstellungen (Datenpfade relativ), `.secret_key`, leere `logs/`, `backups/`, `druckausgabe/`, `UMZUG_LIESMICH.txt`, `daten/umzug.json`; optional Sicherungen. Schreiben als `.teil`, `testzip`, umbenennen; bei Fehler Sperre zurück. Die Middleware in `main.py` liest die Sperre bei jedem Schreibzugriff aus der Datenbank und beantwortet POST/PUT/PATCH/DELETE mit 423 (Seite bzw. JSON für API und Scanner-Sync, der die Offline-Buchungen behält), außer Anmelden, Abmelden, Paket erstellen und Sperre aufheben. `windows/UMZUG_ABSCHLIESSEN_als_Admin.bat`: Programmprüfung, Verknüpfung, Dienst und Firewall.
- **Testsystem** (`app/testsystem.py`, `windows/TESTSYSTEM_STARTEN.bat`): eine zweite Instanz mit `LV_HOME=testsystem`, nur `http://localhost:8090`, ohne Sicherung, ohne Mail, mit eingeschaltetem CMMS. Die Datenbank wird per SQLite-Backup aus dem Echtbetrieb kopiert, auf Wunsch neu. Fotos und Dokumente werden nicht kopiert. Der Ordner `testsystem` gehört nie in ein Update-Paket.
- **Asset-Import** (`app/services/asset_import.py`): Er liest CSV (UTF-8 oder Windows-1252, Trennzeichen erkannt, SharePoint-Schemazeile übersprungen) und Excel (Blatt automatisch oder benannt), bis 10 MB und 20.000 Zeilen. Die Zeilen liegen bis zur Übernahme in `ih_import_zeilen`.
  - `planen()` schreibt nichts. Es ermittelt neu/geändert/unverändert/Fehler und die anzulegenden Ebenen.
  - `ausfuehren()` schreibt in der Transaktion des Aufrufers. Der Testlauf rollt sie zurück.
  - Die Übernahme ist nur mit der Kennung der letzten Prüfung möglich: Zuordnung, Optionen und Stand von `ih_plaetze`.
  - Schlüssel ist `ih_plaetze.schluessel`, die Tag-Nr. ohne Leerzeichen und kleingeschrieben.
  - Links werden nur mit `http(s)://` gespeichert und mit `rel="noopener noreferrer"` geöffnet.

Datensicherung

Ablauf je Termin: SQLite-Backup-API kopiert die laufende Datenbank in eine temporäre Datei; Buchungen laufen dabei weiter. Daraus wird ein **Fingerabdruck** (SHA-256 über alle Tabellen und die Liste der Anhänge mit Größe und Änderungszeit) berechnet. Nicht mitgezählt werden der Sicherungsstatus in `settings` (`backup_*`, `letztes_backup`, `letzte_meldemail`) und `users.letzter_login`. Die ZIP-Datei wird einmal gebaut und in jeden Speicherort kopiert: zuerst als `….zip.teil`, mit `fsync` auf den Datenträger geschrieben, per CRC-Prüfung (`testzip`) kontrolliert und erst dann umbenannt. Bei „nur bei Änderung“ wird ein Speicherort übersprungen, wenn dort die Datei mit demselben Fingerabdruck noch vorhanden ist. Ein neuer oder geleerter Speicherort bekommt also sofort eine Sicherung.

Zustand je Speicherort (letzte erfolgreiche Prüfung, Datei, Größe, freier Platz, letzter Fehler) steht als JSON in `settings.backup_status`, der zuletzt erledigte Termin in `settings.backup_termin`. Ein fehlgeschlagener Speicherort wird alle 30 Minuten gezielt erneut versucht; die anderen werden dabei nicht neu beschrieben. Warnung auf der Startseite (nur Administratoren): letzter Versuch fehlgeschlagen, keine Sicherung vorhanden, letzte Sicherung älter als 26 Stunden, freier Platz unter dem Dreifachen der letzten Sicherung. Aufräumen: Dateien `lager_backup_*.zip` älter als die Frist, die neuesten 3 bleiben immer.

Wiederherstellen (`/einstellungen/backup-wiederherstellen`, nur Administratoren, Passwort + Bestätigung, Fehlversuche zählen zur Anmeldesperre): Prüfung der ZIP (CRC, keine Pfade außerhalb von `anhaenge/`, `PRAGMA integrity_check`, Pflichttabellen, mindestens ein aktiver Administrator) **vor** jeder Änderung → Sicherung des aktuellen Stands als `vor_wiederherstellung_….zip` → Anhänge in einen Nachbarordner entpacken → Datenbank per SQLite-Backup-API in die laufende Datenbank kopieren (WAL-Modus bleibt, keine Reste alter `-wal`-Dateien, wartende Buchungen laufen danach weiter) → Verbindungen neu öffnen, fehlende Tabellen älterer Versionen ergänzen → Anhangsordner tauschen → Eintrag im Protokoll. Der Sicherungsstatus des PCs bleibt erhalten.

Ordnerauswahl: `/einstellungen/backup-ordner` listet Laufwerke (Windows: `GetLogicalDrives`, Laufwerkstyp und -name) bzw. Unterordner ohne versteckte/Systemordner – so, wie der Prozess der Lagerverwaltung sie sieht (beim Dienst: NETZWERKDIENST).

Datenmodell

`artikel`, `lagerplaetze`, `bestand` (Menge je Artikel und Platz), `bewegungen` (jede Buchung mit Zeit, Benutzer, Menge ±, Bestand danach, Empfänger, Kostenstelle, Zweck, Quelle pc/mobil/offline/api/import; ab 2.5.0 auch Preis und Auftrag), `reservierungen`, `bestellungen`, `inventuren`/`inventur_pos`, `lieferanten`, `anhaenge`, `users`, `audit`, `settings`, `sync_log`; ab 2.5.0 `schema_version`, `benutzer_rollen`.

Instandhaltung: `ih_plaetze` ist der technische Platz mit `art` = `bereich` | `unit` | `platz` und der Hierarchie über `eltern_id`. Weitere Felder: Code (Tag-Nr.), Schlüssel, Bezeichnung, Bauteilklasse, PLT-Sicherheitseinrichtung, Kritikalität A/B/C, Hersteller, Modell, Seriennummer, Lieferant, Status, Inbetriebnahme, Anschaffung, Bestell- und Leistungsnummer, Kosten, Doku-Link, Notiz und `zusatz` (JSON mit weiteren Spalten aus dem Import). Dazu kommen `ih_import_laeufe` und `ih_import_zeilen` (Importläufe mit Zuordnung, Ergebnis und Meldungen je Zeile). Anlagen werden nicht gelöscht, nur archiviert (`aktiv`). Wartung (ab 2.6.0): `ih_massnahmen`, `ih_klassen_intervalle`, `ih_kampagnen` (auch Pflichtprüfungen mit Rechtsgrundlage, Durchführung, Vorlauf), `ih_auftraege`, `ih_auftrag_positionen` (Protokoll WP-01/WP-02: Zustand, Schritte/Messwerte als JSON, Ergebnis, Befund, Abschlusskontrolle, SIL, durchgeführt von/am), `ih_statuslog`, `ih_dokumente`. Ab 2.7.0: `ih_kampagne_bauteile` (einzelne Bauteile eigener Kampagnen), `ih_stueckliste` (Ersatzteile je Bauteil).

Sichtlagerboxen sind eigene Zeilen in `lagerplaetze` mit dem Code des Platzes plus `-B<Nummer>` (z. B. `C1-R2-3-B4`). Die Zugehörigkeit zum Platz ergibt sich aus dem Code (`queries.box_eltern`), es gibt keine zusätzliche Spalte und keine Datenbankänderung. Buchen, Scannen, Etiketten, Inventur, Sicherung und Import behandeln Boxen wie jeden Platz. Umbenennen eines Platzes benennt seine Boxen mit (auch in `bewegungen.lagerplatz`). Nachbestell-Markierung (`queries.nachbestell_plaetze`): alle Plätze mit einer Bestandszeile eines aktiven Artikels im Status „melden“ oder „kritisch“, wie unter *Nachbestellung*. Die Farbe Weinrot ist als `--wein` in `frontend/input.css` hinterlegt.

Mengen: Komma oder Punkt als Dezimaltrenner; „nan“, „inf“ und Werte über 1 Milliarde werden abgelehnt.

Buchungsarten: `eingang`, `ausgang`, `umbuchung` (zwei verknüpfte Zeilen), `inventur`, `anlage`, `ausleihe`, `rueckgabe`, `info` (nur Historie aus dem Altsystem).

Alle Bestandsänderungen laufen über `app/services/lager.py` (Klasse `Lager`).

Übernahme aus Casper

`app/services/casper_import.py` liest den HeidiSQL-Export (nur INSERT-Anweisungen am Zeilenanfang – das `INSERT` im Casper-Trigger `bewegungsdaten_after_insert` wird ignoriert) und übernimmt:

Casper	Neu
stammdaten.Artikelnummer / Bezeichnung	artikel.nummer / bezeichnung
Meldebestand / Mindestbestand (Text, 0 = nicht gepflegt)	Zahl oder leer
Freifeld1 „Lieferant“	lieferanten (Schreibvarianten zusammengeführt)
Freifeld2 „Gruppe“, Freifeld3 „Typenbezeichnung“	kategorie, typ
Freifeld4 „Austragungsgrund“, Freifeld5	notiz
lagerorte (Leerzeichen bereinigt, Duplikate summiert, leer → OHNE-PLATZ)	lagerplaetze + bestand
bewegungsdaten (Eingang, Ausgang, Umbuchung/Umlagerung, Inventur, Neuer Artikel, Artikel bearbeitet/gelöscht)	bewegungen (quelle = import)
benutzer	users (gesperrt, ohne Passwort)

Lagerort-Zeilen zu nicht mehr vorhandenen Artikeln werden nicht übernommen (im Bericht genannt).

Schnittstelle

Token-Authentifizierung (`Authorization: Bearer <Schlüssel>`, Schlüssel je Benutzer unter *Einstellungen → Schnittstelle*). Endpunkte unter `/api/v1/…`: Artikel (Liste, Detail mit Beständen je Platz), Lagerplätze, Bewegungen seit ID, Buchung anlegen, Reservierung anlegen. Beschreibung maschinenlesbar unter `/api/openapi.json`. Gedacht für die spätere Anbindung der Instandhaltungs-Software (Ersatzteile reservieren/entnehmen je Auftrag).

Entwicklung

```
pip install -r requirements.txt
cd frontend && npm install && ./build_css.sh
python -m app.main                      # http://localhost:8080 (nur lokal), https://<PC>:8443
python -m pytest tests                  # LV_CASPER_EXPORT=<datei.sql> testet zusätzlich die
Übernahme echter Daten (Sollwerte 821 / 377 / 2702)
```